

Hadamard-Based Rejection Sampling for Fair Dice: A Low-Depth Quantum Circuit Model for Tabletop Games

Önder Öztürk

Abstract—We present a practical method for producing mathematically fair six-sided dice rolls from quantum measurements. Each die is encoded with three qubits. Applying $H^{\otimes 3}$ to the all-zero state produces a uniform superposition over eight basis states. By rejecting the outcomes 0 and 7 and conditioning on the valid set $\{1, \dots, 6\}$, the resulting distribution is exactly uniform ($1/6$ per face). For two dice (as required by backgammon and its variants such as Tavla), the construction extends to a six-qubit product circuit $H^{\otimes 6}$ whose marginals remain independent. The same circuit is exposed through a thin HTTP service that can dispatch to either the Qiskit Aer simulator or IBM Quantum hardware; the game engine consumes only the final classical values together with a provenance/fallback flag. We prove uniformity via elementary conditional probability, demonstrate the bias introduced by the common modulo mapping, and report reproducible (seeded) Aer experiments ($N = 50,000$ valid rolls) whose marginal and 6×6 joint face distributions are consistent with the predicted uniform law (marginal $\chi^2 = 10.1$ on 5 degrees of freedom (d.o.f.), $p = 0.07$; joint $\chi^2 = 26.8$ on 35 d.o.f., $p = 0.84$). Because the Aer simulator performs ideal Born sampling, these experiments validate the sampling pipeline rather than the abstract fairness guarantee, which is already exact. Runs on two real IBM Quantum processors (4000 shots each) exhibit the expected percent-level noise bias (post-rejection TVD between 0.019 and 0.040 from uniform); applying matrix-free readout-error mitigation reduces the bias on the noisier device by roughly a factor of two, with both devices converging to $\text{TVD} \approx 0.021$. The approach uses a single layer of Hadamard gates and incurs negligible rejection overhead, making it suitable for low-latency integration in consumer games while cleanly separating quantum sampling from classical game logic.

Index Terms—Quantum random number generation, quantum rejection sampling, Hadamard circuit, fair dice, Qiskit, IBM Quantum, readout error mitigation, backgammon, tabletop games.

I. INTRODUCTION

FAIR random number generation is fundamental to the integrity of dice-driven games. In backgammon and its regional variants (collectively “Tavla”), the joint outcome of two independent six-sided dice directly determines the legal move set. Even small biases in the dice distribution can, over thousands of games, produce measurable advantages for one player or strategy class.

Ö. Öztürk is with the Information Technology Department, Rectorate, Kültür Health Sciences University, Türkiye (ORCID: 0000-0001-6460-9497).

A reproducibility package—source code, seeded data, and manuscript sources—is openly available at <https://github.com/onder-ozturk/quantum-dice> (MIT-licensed).

Classical pseudorandom generators (PRNGs) or operating-system entropy sources are adequate for most entertainment software. Nevertheless, there is growing interest in *verifiable* or *externally auditable* randomness, especially in online or blockchain-adjacent gaming, educational demonstrations of quantum computing, and marketing of “genuinely quantum” experiences. Cloud-accessible quantum processors [1] now make it feasible to obtain measurement samples from explicitly constructed quantum circuits and to expose them through ordinary web APIs.

A naïve approach—prepare n qubits in uniform superposition with $H^{\otimes n}$ and reduce the resulting integer modulo 6—introduces a well-known structural bias: certain faces receive twice the probability of others. We instead adopt *rejection sampling*: the two forbidden outcomes are discarded and the conditional distribution over the remaining six is exactly uniform. The construction is textbook quantum mechanics (uniform superposition) combined with elementary probability, yet it yields a clean, testable, and low-depth implementation that has seen surprisingly little rigorous treatment in the quantum-gaming literature.

Our contributions are:

1. A three-qubit circuit plus rejection rule that realises an exactly fair six-sided die, together with the corresponding two-die product construction (Section III–V).
2. An explicit proof that the conditional distribution is uniform and a side-by-side comparison with the biased modulo mapping (Section IV).
3. A production-grade service architecture that preserves the identical mathematical contract on both simulator and real hardware while providing transparent fallback (Section VIII).
4. Large-scale experimental validation on the Qiskit Aer simulator, including χ^2 goodness-of-fit tests, total-variation distances, and empirical confirmation of the per-shot acceptance probability (Section IX).
5. A multi-backend hardware run on two IBM Quantum processors with matrix-free readout-error mitigation, quantifying the residual percent-level bias and how much of it is removable (Section IX–D).

Related work includes the well-known IBM “Truly Quantum Dice” demonstration [2], the quantum random-number-generation surveys of Herrero-Collantes and Garcia-Escartin [3] and Ma et al. [4], and the circuit construction of Orts et al. [5] for sampling integers inside an arbitrary

interval—of which our concrete $d = 6$ rule is a special case. Quantum games have previously been realised on a photonic one-way quantum computer [6] and on IBM-Q [7]. Classical treatments of unbiased sampling from biased sources date back to von Neumann [8], are discussed in detail by Knuth [9], and are codified for random-bit-generator constructions by NIST [10]. A strictly stronger goal—cryptographically *certified* randomness from quantum sampling—was put on rigorous footing by Aaronson and Hung [11] and recently demonstrated experimentally on a 56-qubit trapped-ion processor [12]; we emphasise that our construction makes *no* certification claim and provides no advantage over a classical generator for ordinary play. Recent benchmarks of gate-based quantum random-number generation on IBM superconducting hardware likewise report usable but low-throughput, high-cost-per-bit randomness [13], consistent with our own observation that a cloud quantum round-trip is, for ordinary dice, strictly more expensive than a local generator. Our emphasis is narrower: a minimal, exactly analysable circuit for the concrete $d = 6$ case that arises in tabletop games, together with an end-to-end engineering contract suitable for mobile deployment.

II. MATHEMATICAL PRELIMINARIES

Definition 1 (Computational basis). *For a single qubit the computational basis states are denoted $|0\rangle, |1\rangle$. An n -qubit system has basis states $|x\rangle$ for $x \in \{0, 1, \dots, 2^n - 1\}$, where the integer x is the binary representation of the bit string.*

The Hadamard gate (see, e.g., [14])

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

creates an equal superposition when applied to a computational-basis state:

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

Applying H to every qubit of the all-zero state yields the uniform superposition

$$H^{\otimes n}|0^n\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle.$$

Measurement of this state returns each integer x with probability exactly $1/2^n$ (Born rule).

III. SINGLE DIE FROM THREE QUBITS

A six-sided die requires six equally likely outcomes. Three qubits naturally produce eight. We therefore interpret the measurement outcome $X \in \{0, \dots, 7\}$ and map it to a die face via rejection sampling rather than reduction modulo 6.

The circuit for one die is simply

$$|\psi_d\rangle = H^{\otimes 3}|000\rangle = \frac{1}{\sqrt{8}} \sum_{x=0}^7 |x\rangle.$$

Consequently $P(X = x) = 1/8$ for each x .

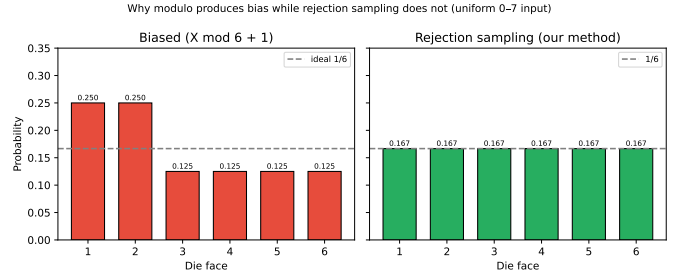


Fig. 1. Comparison of the biased modulo mapping with the exactly uniform distribution obtained by rejection sampling. Both start from a uniform random variable on $\{0, \dots, 7\}$.

IV. REJECTION SAMPLING AND EXACT FAIRNESS

Define the valid set $A = \{1, 2, 3, 4, 5, 6\}$. A shot is accepted only when the raw outcome lies in A ; otherwise it is rejected and the next shot in the batch is examined.

Proposition 1 (Exact fairness of a single die). *Let X be uniform on $\{0, \dots, 7\}$. Define the random variable $D = X \mid X \in A$. Then $P(D = k) = 1/6$ for every $k \in A$.*

Proof. By definition of conditional probability and the uniformity of X ,

$$P(D = k) = P(X = k \mid X \in A) = \frac{P(X = k)}{P(X \in A)} = \frac{1/8}{6/8} = \frac{1}{6}.$$

□

In contrast, the popular “modulo” mapping $D = (X \bmod 6) + 1$ produces a visibly biased distribution:

$$P(D = 1) = P(D = 2) = \frac{2}{8}, \quad P(D = 3) = \dots = P(D = 6) = \frac{1}{8}.$$

Figure 1 juxtaposes the two distributions.

The rejection rule is realised in a few lines of classical post-processing after measurement; see the `pick_valid_dice` routine in the accompanying implementation.

Table I situates the construction against the relevant prior art. We stress the honest conclusion it encodes: on fairness the method is *equivalent* to textbook classical rejection sampling and offers no statistical advantage over a cryptographically secure pseudo-random generator for ordinary play. Its only distinguishing property is that the underlying bits originate in a measured quantum state, giving auditable “quantum” provenance—at the cost of a network round-trip and, on real hardware, residual noise (Section IX-D).

V. TWO DICE AND INDEPENDENCE

Tavla (and standard backgammon) requires two independent dice. We encode both dice in a single six-qubit circuit:

$$|\Psi\rangle = H^{\otimes 6}|000000\rangle = \frac{1}{8} \sum_{x=0}^7 \sum_{y=0}^7 |x\rangle|y\rangle.$$

The first three qubits represent die 1 and the second three represent die 2. Because the state is a tensor product, the two registers are independent even before measurement. After applying the per-die rejection rule we obtain a pair (D_1, D_2) whose joint distribution is exactly uniform over the 6×6 grid.

TABLE I

WHERE THIS CONSTRUCTION SITS RELATIVE TO PRIOR ART. “FAIR $d6$ ” IS EXACT UNIFORMITY IN THE IDEAL (NOISE-FREE) MODEL. THE PRESENT METHOD MATCHES CLASSICAL REJECTION SAMPLING ON FAIRNESS; THE QUANTUM VARIANTS PAY A NETWORK ROUND-TRIP AND INHERIT DEVICE NOISE.

Method	Fair $d6$?	Bit source	Overhead vs CSPRNG
$H^{\otimes 3}$ then $(\cdot \bmod 6)$	no (biased)	quantum	—
$H^{\otimes 3}$ + rejection (<i>this work</i>)	yes (ideal)	quantum	higher (network, noise)
von Neumann/Knuth rejection [8], [9]	yes	classical	none (baseline)
Arbitrary-interval circuit [5]	yes (general d)	quantum	higher
CSPRNG + rejection [10]	yes	classical	cheapest

Proposition 2 (Joint uniformity of two dice). *Let $D_1 = X \mid X \in A$ and $D_2 = Y \mid Y \in A$. Then for every $i, j \in A$,*

$$P(D_1 = i, D_2 = j) = \frac{1}{36}.$$

Proof. The six-qubit state factors, therefore X and Y are independent and each uniform on $\{0, \dots, 7\}$. The event that both lie in A has probability $(6/8)^2 = 9/16$. The conditional probability of any particular ordered pair is therefore

$$\frac{(1/64)}{(36/64)} = \frac{1}{36}.$$

□

VI. GENERALISATION AND BATCH SUCCESS PROBABILITY

For m dice the circuit uses $3m$ qubits and the per-shot acceptance probability is

$$p_m = \left(\frac{6}{8}\right)^m = \left(\frac{3}{4}\right)^m.$$

The expected number of shots until the first valid m -tuple is $\mathbb{E}[T_m] = 1/p_m = (4/3)^m$. For the two-dice case of interest, $\mathbb{E}[T_2] = 16/9 \approx 1.78$ raw shots (well below the batch size of 64 used in production). The probability that a complete batch of $B = 64$ shots contains *no* valid pair is

$$(1 - p_2)^{64} = \left(\frac{7}{16}\right)^{64} \approx 1.5 \times 10^{-23},$$

which is negligible for all practical purposes.

The same recipe gives a fair d -sided die for any d : take $k = \lceil \log_2 d \rceil$ qubits, accept the first d of the 2^k equiprobable outcomes and reject the remaining $2^k - d$. Table II lists the standard tabletop and role-playing dice; powers of two ($d = 4, 8$) need no rejection, while $d = 6, 10, 12, 20$ carry only a modest overhead.

VII. ALGORITHM

The end-to-end procedure executed by the service (identical for Aer and IBM) is:

```
Input: m (dice), B (shots per batch, default 64)
n <- 3 * m
Build n-qubit circuit, n classical bits
Apply H to every qubit
Measure every qubit
for up to 4 batches:
    run circuit for B shots (memory=True)
```

TABLE II

GENERALISATION TO A FAIR d -SIDED DIE. $k = \lceil \log_2 d \rceil$ QUBITS GIVE 2^k EQUIPROBABLE OUTCOMES; THE FIRST d ARE ACCEPTED AND THE OTHER $2^k - d$ REJECTED, SO THE PER-DIE ACCEPTANCE PROBABILITY IS $p = d/2^k$ AND THE EXPECTED NUMBER OF RAW SHOTS PER ACCEPTED VALUE IS $1/p = 2^k/d$.

d	qubits k	states 2^k	rejected	$\mathbb{E}[\text{shots}] = 2^k/d$
4	2	4	0	1.00
6	3	8	2	1.33
8	3	8	0	1.00
10	4	16	6	1.60
12	4	16	4	1.33
20	5	32	12	1.60

for each bitstring in result memory:

```
    decode m groups of 3 bits -> integers in 0..7
    if all integers are in 1..6:
        return the m-tuple as dice
return secure classical fallback (last resort)
```

The quantum portion is a single layer of $3m$ Hadamard gates followed by $3m$ single-qubit measurements (gate depth 1 in the ideal parallel model; the measurement adds one further layer). Classical post-processing per batch costs $O(Bm)$, and for $m = 2$ a single batch of 64 shots almost always suffices.

VIII. SERVICE ARCHITECTURE AND CONTRACT

The quantum sampling layer is deliberately isolated from the game engine. A lightweight FastAPI micro-service exposes a single endpoint

```
GET /roll?backend=aer|ibm&dice=2.
```

The response always contains the classical values ‘d1’, ‘d2’, the actual backend that served the result, a job identifier, the number of shots consumed, and a Boolean ‘fallback’ flag. When the client requests the IBM backend but the job times out or the hardware is unavailable, the service transparently returns an Aer result with `fallback=true`; a final last-resort path uses a classical cryptographic RNG. Consequently a roll requested as “quantum” may in practice be served by the simulator or by a classical generator: the explicit backend and fallback fields make this provenance auditable, but a genuine quantum-hardware origin is best-effort, not guaranteed. An application that must guarantee hardware provenance should replace this synchronous, fallback-on-timeout endpoint with an *asynchronous* submit/poll interface that returns only genuine hardware results—accepting seconds-to-minutes of

queue latency—and should never label a fallback roll as “quantum.” The synchronous design here deliberately trades that guarantee for the low, bounded latency a real-time game needs. The Android game engine never sees quantum state vectors; it receives ordinary integers that are fed directly into the classical Tavla transition function.

Figure 2 depicts the data flow and the mathematical invariant that is preserved across both back-ends.

IX. EXPERIMENTAL VALIDATION

All experiments use the exact production circuit (`build_dice_circuit`) and the same accept test as the production `pick_valid_dice`; the run is seeded (`seed_simulator= 42`) and is therefore bit-for-bit reproducible from the accompanying scripts. We collected $N = 50,000$ valid two-dice rolls on the Qiskit Aer simulator [15], corresponding to 100,000 individual die faces.

A. Marginal uniformity

The observed face counts were

$$(16521, 16600, 16783, 16701, 16446, 16949).$$

A χ^2 goodness-of-fit test (5 degrees of freedom, no estimated parameters) against the uniform expectation of 16,666.67 per face yields

$$\chi^2 = 10.13, \quad p = 0.072, \quad \text{TVD} = 0.0043.$$

The test does not reject uniformity at the 5% level; the largest single-face residual is +282 (about 2.4σ). Because the Aer simulator draws from precisely the Born distribution that Propositions 1–2 already guarantee, this experiment is best read as a sanity check on the (pseudo-random) sampling and bit-decoding pipeline, not as an empirical test of fairness on a physical device; a high p -value is failure to reject, not positive confirmation. Figure 3 shows the observed versus expected counts.

B. Joint uniformity

The 6×6 contingency table of (d_1, d_2) pairs was tested for joint uniformity over the 36 cells. The χ^2 statistic on 35 degrees of freedom is 26.85 with $p = 0.84$ and total-variation distance 0.0097 from the ideal $1/36$ per cell, again consistent with the predicted distribution. We stress that independence of the two dice holds *a priori* from the tensor-product structure of $|\Psi\rangle$ (Section V); this table is therefore a joint-uniformity goodness-of-fit check, not an empirical independence test, and on an ideal simulator it cannot reveal the qubit crosstalk that a real device might exhibit.

C. Acceptance rate

Direct sampling of 200,000 raw shots produced 112,687 valid pairs, giving an empirical per-shot acceptance probability of 0.56344, in close agreement with the theoretical value $(6/8)^2 = 0.5625$. The 50,000 analysed rolls are the first 50,000 accepted pairs of this same seeded run; returning the first accepted shot of a batch is unbiased because the raw shots

are independent and identically distributed, so conditioning on “first accepted” preserves the uniform conditional law. With the production batch size $B = 64$ the expected number of raw shots per valid pair is $\mathbb{E}[T_2] = 16/9 \approx 1.78$, and the probability that a batch contains no valid pair is $(7/16)^{64} \approx 10^{-23}$ (Section V).

D. Multi-backend hardware run with readout-error mitigation

To probe how the ideal-circuit guarantee degrades on physical devices—and how much of that degradation is removable—we executed the identical hadamard-6q circuit on *two* independent IBM Quantum processors, `ibm_marrakesh` and `ibm_kingston` (both 156-qubit Heron devices), with 4000 shots each (8000 raw die values per backend). On each backend we additionally applied M3 matrix-free measurement-error mitigation [16] (the `mthree` package), calibrated in situ with 4000 shots on the six measured qubits, and recomputed the conditioned distribution from the mitigated quasiprobabilities. The per-job identifiers and full distributions are recorded in the released data.

Table III summarises the result and Figure 4 visualises it. The two backends differ markedly in raw readout bias. On `ibm_kingston` the conditioned 1–6 distribution is clearly skewed—TVD = 0.040 from the ideal $1/6$, with $\chi^2 = 42.3$ (5 d.o.f., $p < 10^{-3}$), which *does* reject uniformity at this sample size—whereas on `ibm_marrakesh` the bias is roughly half as large (TVD = 0.019, $\chi^2 = 7.2$, $p = 0.20$, not rejected). The empirical per-shot acceptance was 0.564 and 0.571 respectively, both close to the theoretical $9/16 = 0.5625$.

Readout-error mitigation behaves exactly as its purpose predicts. On the noisier device (`ibm_kingston`) M3 nearly halves the bias, from TVD = 0.040 to 0.021; on the cleaner device (`ibm_marrakesh`), where the raw bias is already at the percent level, mitigation marginally *increases* the TVD (from 0.019 to 0.021), because the finite-shot sampling noise of the calibration exceeds the small readout error it removes. After mitigation both backends converge to essentially the same residual, TVD ≈ 0.021 , indicating that the leftover bias is dominated by effects M3 does not address (state-preparation error and a residual, non-readout error floor) rather than by measurement error.

Two structural points follow, independent of the device. First, rejection sampling removes the *structural* modulo bias but cannot remove *hardware* bias: any skew already present in the raw 0–7 samples is inherited by the conditional distribution, and readout mitigation only partially recovers it. Second, the residual unfairness on current hardware is at the percent level—small, but on `ibm_kingston` large enough to be statistically detected; a deployment that advertises “quantum dice” should therefore publish per-backend calibration and, where it matters, apply readout-error mitigation, rather than rely on the ideal guarantee.

The circuit diagram actually executed is shown in Figure 5; the rejection flow is illustrated in Figure 6.

X. DISCUSSION

The mathematical fairness guarantee holds only for the ideal circuit. On real IBM hardware, readout errors, decoherence,

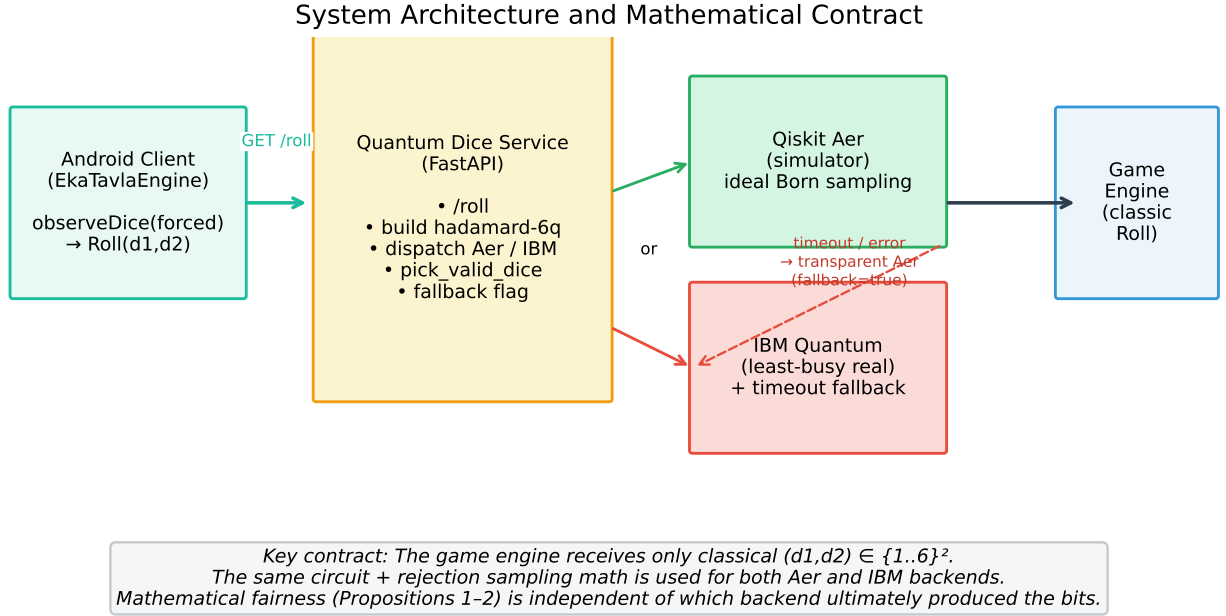


Fig. 2. Service architecture. The identical ‘hadamard-6q’ circuit and rejection-sampling post-processor are used for both simulator and real hardware. The game engine receives only classical die values together with provenance metadata.

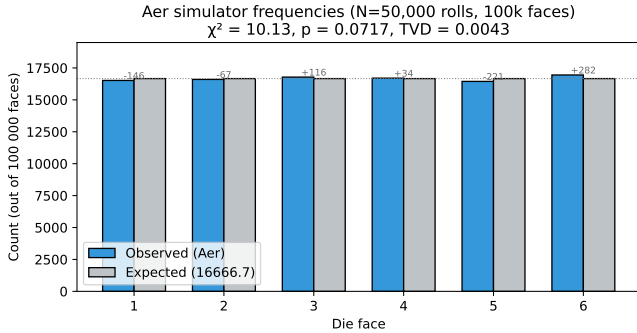


Fig. 3. Observed face frequencies from 50,000 Aer rolls (100,000 die faces). Residuals are at the level of statistical fluctuation; the χ^2 test does not reject uniformity.

TABLE III
TWO-BACKEND HARDWARE RUN (4000 SHOTS EACH) OF THE HADAMARD-6Q CIRCUIT, BEFORE AND AFTER M3 READOUT-ERROR MITIGATION [16]. TVD IS THE TOTAL-VARIATION DISTANCE OF THE REJECTION-CONDITIONED 1–6 DISTRIBUTION FROM THE IDEAL 1/6; THE χ^2 (5 D.O.F.) AND ACCEPTANCE REFER TO THE RAW RUN.

Backend	raw TVD	mit. TVD	raw χ^2 (p)	accept.
ibm_marrakesh	0.019	0.021	7.2 (0.20)	0.564
ibm_kingston	0.040	0.021	42.3 ($< 10^{-3}$)	0.571

and gate infidelity will perturb the raw 0–7 distribution before rejection. Because rejection discards two out of eight outcomes, any bias already present in the hardware samples is inherited by the conditional distribution (measured in Sec-

tion IX-D: TVD from 0.019 to 0.040 across two backends, only partially recoverable by readout-error mitigation). Consequently, production deployments that advertise “quantum dice” should either (a) remain on the simulator, (b) publish per-backend calibration data together with the observed TVD, or (c) apply additional classical conditioning/extraction. In our service the ‘backend’ and ‘fallback’ fields make the provenance of every roll explicit, allowing downstream auditing.

From a systems perspective the construction is deliberately over-engineered for a mobile game: a remote round-trip, possible queueing, and quota consumption are all incurred for randomness that a modern phone can generate locally at essentially zero cost and with cryptographic strength. The value of the present work therefore lies less in “better dice” than in (i) a clean pedagogical example of mapping a continuous quantum probability distribution onto a discrete uniform one, (ii) a concrete, reproducible demonstration of cloud quantum computing inside a consumer application, and (iii) an existence proof that the engineering surface between a quantum backend and a classical game engine can be reduced to a few dozen lines of well-specified code.

Future extensions could include device-independent certified randomness protocols, on-device quantum simulators for zero-latency play, or generalisation to other dice geometries (e.g., 4-, 8-, 10-, 12-, 20-sided) with optimal qubit counts and minimal rejection overhead.

a) Limitations. Three limitations bound the present claims. (i) The hardware evidence is a single run on each of two backends (Section IX-D); the readout/gate-noise bias

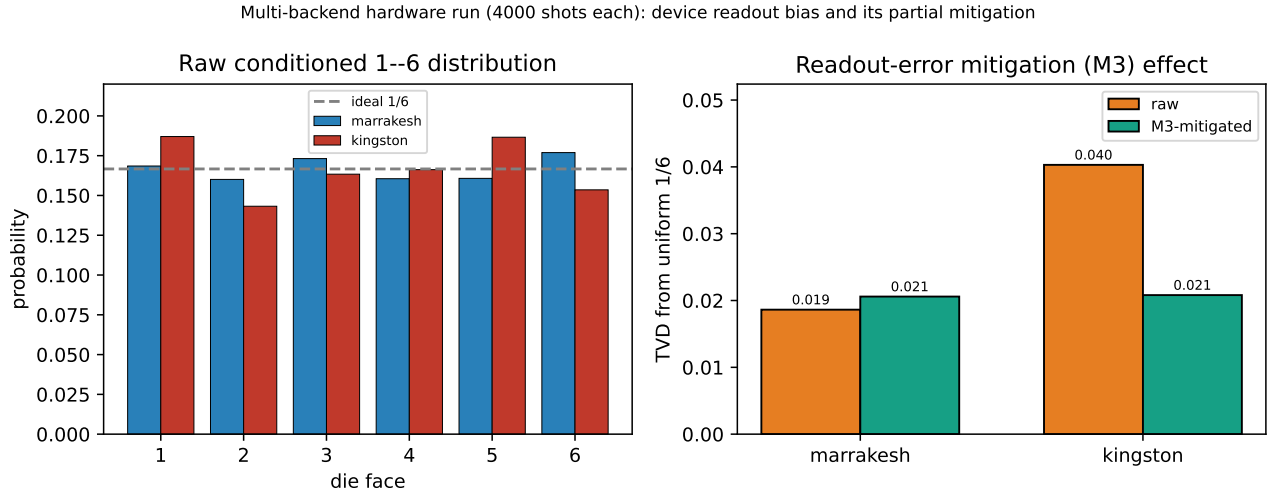


Fig. 4. Multi-backend hardware run (4000 shots each on the 156-qubit Heron devices `ibm_marrakesh` and `ibm_kingston`). Left: the raw rejection-conditioned 1–6 distribution per backend against the ideal 1/6; `ibm_kingston` is visibly more skewed. Right: total-variation distance from uniform before and after M3 readout-error mitigation [16]. Mitigation nearly halves the bias on the noisier device but slightly increases it on the cleaner one, where calibration sampling noise dominates; both converge to $\text{TVD} \approx 0.021$.

we observe should be characterised across more devices and repeated runs to estimate run-to-run variance, and the effect of qubit crosstalk on the product-state independence assumption studied—which we leave to future work. (ii) The exactness result concerns the ideal circuit; under the fallback path a roll may be served by the simulator or a classical RNG, so end-to-end “quantum” provenance is best-effort. (iii) The contribution is a clean engineering-and-pedagogy construction rather than a new quantum primitive—for ordinary play a local CSPRNG is cheaper and cryptographically stronger, and the method provides no randomness *certification* in the device-independent sense.

XI. CONCLUSION

We have shown that a single-layer Hadamard circuit on three qubits per die, followed by the rejection of two out of eight measurement outcomes, yields a mathematically exact uniform six-sided die. The construction extends without modification to any number of independent dice via a product state. Reproducible simulator experiments are consistent with the predicted statistics, with the caveat that an ideal simulator validates the sampling pipeline rather than the fairness guarantee itself. By exposing the sampler through a narrow, auditable HTTP contract we separate the quantum sampling concern from the classical game logic; the same contract also carries hardware-sourced bits, and runs on two real IBM processors confirm that the guarantee degrades gracefully (percent-level bias) under device noise, with matrix-free readout-error mitigation recovering much of the bias on the noisier device; a larger multi-device characterisation with repeated runs remains future work.

The implementation, the seeded experimental data, the generation scripts, and the figure sources are included in the repository [17]; the seeded run makes every reported number reproducible bit-for-bit.

REPRODUCIBILITY

All source code, the seeded experimental data, the analysis and figure scripts, and the manuscript sources are openly available [17] (tag `v1.0`, MIT-licensed). The simulator results are bit-for-bit reproducible: `generate_experiments.py` runs the exact production circuit with `seed_simulator=42`. The hardware runs used Qiskit Runtime SamplerV2 at the default resilience level, `optimization_level=1` with the standard translator translation stage, and 4000 shots per backend; `run_ibm_experiment.py` performs the single-backend run, and `run_ibm_mitigated.py` performs the two-backend run with M3 readout-error mitigation (`mthree`, calibrated with 4000 shots). Each script records the backend name, job id, and the raw per-shot outcomes. Because hardware calibration drifts, exact device numbers are run-specific; the scripts and the saved JSON allow the procedure to be replayed on any backend.

ACKNOWLEDGEMENTS

We thank the Qiskit open-source community; the Aer simulator made the reproducible experimental campaign straightforward, and the IBM Quantum Runtime primitives underpin the hardware path. We used the `mthree` package for matrix-free measurement-error mitigation. The Android client and 3-D tabletop visualisation layers of Quantum Eka Tavla are outside the scope of this paper but provided the original motivation for a verifiable dice source.

REFERENCES

- [1] IBM Quantum, “Ibm quantum platform,” <https://quantum.ibm.com/>, 2024, cloud access to superconducting quantum processors; least-busy scheduling and Qiskit Runtime.
- [2] R. Huffman, “Truly quantum dice,” <https://medium.com/design-ibm/truly-quantum-dice-cfe372f4c586>, 2018, IBM Design blog post demonstrating Hadamard-based quantum dice on IBM Quantum systems.
- [3] M. Herrero-Collantes and J. C. Garcia-Escartin, “Quantum random number generation,” *Reviews of Modern Physics*, vol. 89, no. 1, p. 015004, 2017.

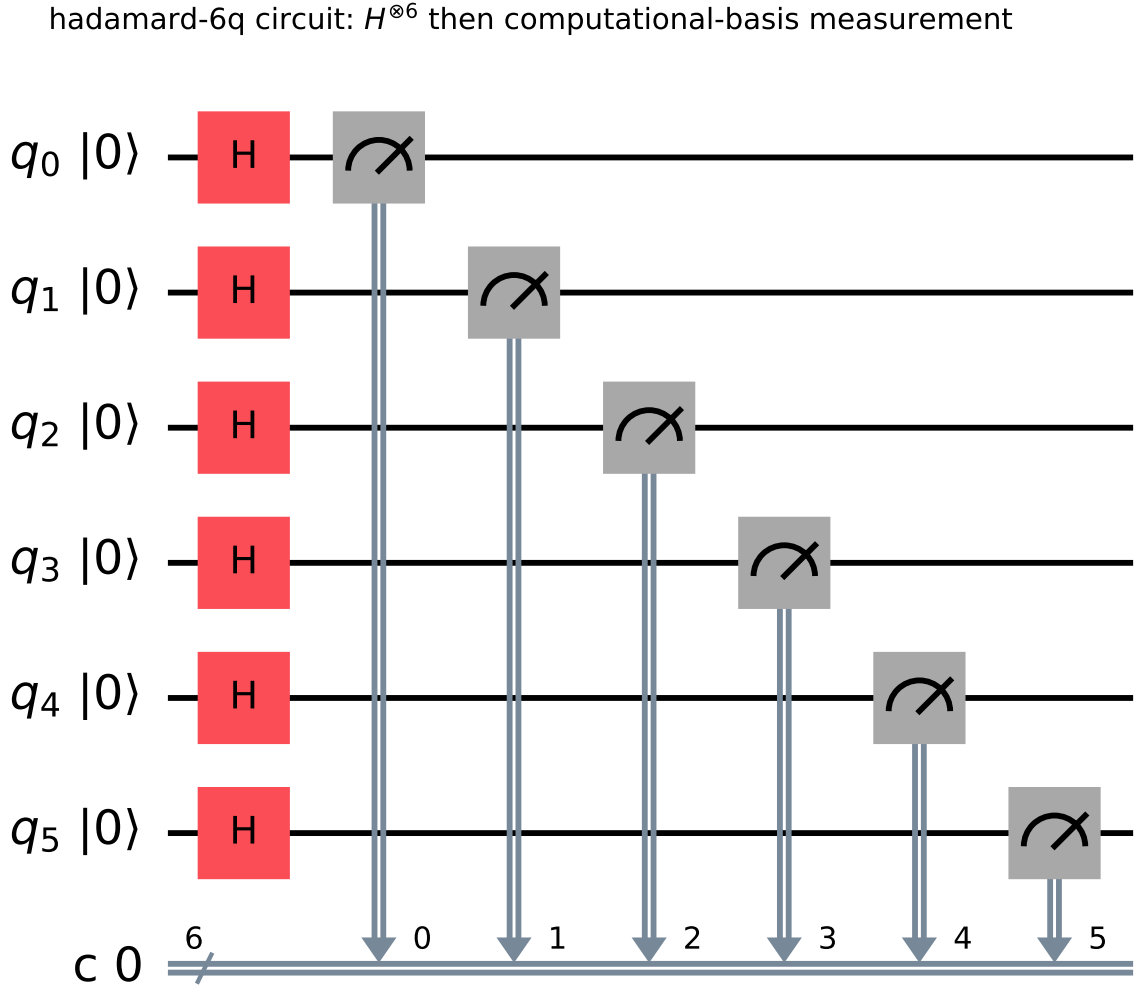


Fig. 5. The six-qubit circuit `hadamard-6q` used for every roll: six Hadamard gates followed by six computational-basis measurements (the classical register is bundled). Qubits q_0 – q_2 encode die 1 and q_3 – q_5 encode die 2; classical post-processing performs the per-die rejection test.

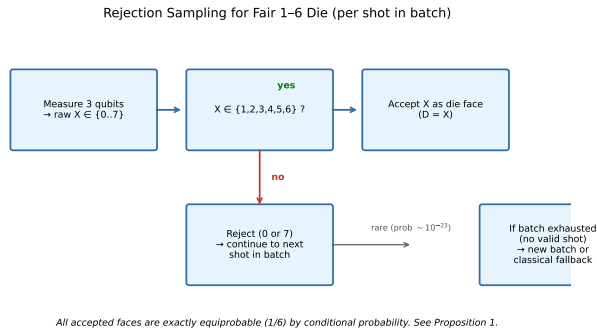


Fig. 6. Rejection sampling decision flow for a single shot. The only quantum operation is the uniform superposition and measurement; the accept/reject test is classical and extremely cheap.

- [4] X. Ma, X. Yuan, Z. Cao, B. Qi, and Z. Zhang, “Quantum random number generation,” *npj Quantum Information*, vol. 2, p. 16021, 2016.
- [5] F. Orts, E. Filatovas, E. M. Garzón, and G. Ortega, “A quantum circuit to generate random numbers within a specific interval,” *EPJ Quantum Technology*, vol. 10, no. 1, p. 17, 2023.

- [6] R. Prevedel, M. Aspelmeyer, Č. Brukner, A. Zeilinger *et al.*, “Experimental realization of a quantum game on a one-way quantum computer,” *New Journal of Physics*, 2007.
- [7] F. Galas, “Quantum games on ibm-q,” in *Proceedings of the 2020 Federated Conference on Computer Science and Information Systems*, 2020.
- [8] J. von Neumann, “Various techniques used in connection with random digits,” *National Bureau of Standards Applied Mathematics Series*, vol. 12, pp. 36–38, 1951, classic paper introducing unbiased bit extraction from biased sources.
- [9] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, 1997, section on random number generation and common pitfalls with modulo bias when mapping to dice.
- [10] NIST, “Recommendation for random bit generator (rbg) constructions,” <https://csrc.nist.gov/publications/detail/sp/800-90c/draft>, 2016, NIST SP 800-90C (draft); discusses conditioning and extraction for random bit generators.
- [11] S. Aaronson and S.-H. Hung, “Certified randomness from quantum supremacy,” *arXiv preprint arXiv:2303.01625*, 2023, protocol for cryptographically certified randomness from sampling-based quantum advantage.
- [12] M. Liu, R. Shaydulin, P. Niroula, M. DeCross *et al.*, “Certified randomness using a trapped-ion quantum processor,” *Nature*, vol. 640, 2025.
- [13] M. M. Louamri, A. Boussahi, N. E. Belaloui, and A. Tounsi, “A study of gate-based and boson sampling quantum random number generation

on ibm and xanadu quantum devices,” *arXiv preprint arXiv:2507.03823*, 2025.

- [14] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.
- [15] IBM Quantum, “Qiskit: An open-source framework for quantum computing,” <https://www.ibm.com/quantum/qiskit>, 2024, version 1.x; documentation and runtime primitives (SamplerV2).
- [16] P. D. Nation, H. Kang, N. Sundaresan, and J. M. Gambetta, “Scalable mitigation of measurement errors on quantum computers,” *PRX Quantum*, vol. 2, no. 4, p. 040326, 2021.
- [17] Ö. Öztürk, “Quantum tavla dice: Service, reproduction scripts, and data,” <https://github.com/onder-ozturk/quantum-dice>, 2026, mIT-licensed reproducibility package: the hadamard-6q circuit and rejection sampling (dice.py), the Aer and IBM runners (backends.py, run_ibm_experiment.py), and the seeded, bit-for-bit reproducible experiment and figure scripts with saved data.